



COURSE CODE/TITLE: CPE010

SECTION: CPE21S1

DATE PERFORMED: August 10, 2026

DATE SUBMITTED: August 10, 2026

NAME: Mendoza, Ryo Shin P.

INSTRUCTOR: Engr. Jimlord Quejado

ACTIVITY NO. 5.1 QUEUE

1. PROCEDURE OUTPUT

ILOA

```
#include <iostream>
#include <queue>

void display(std::queue<char> r);

int main()
{
    std::queue<char> q;

    q.push('c');
    q.push('p');
    q.push('e');
    q.push('0');
    q.push('1');
    q.push('0');

    std::cout << "=====\n";
    std::cout << "Queue Implementation\n";
    std::cout << "=====\n\n";
    // Display the queue
    std::cout << "Display: ";
    display(q);
    std::cout << "=====\n\n";
```

```
// empty()
if (q.empty())
    std::cout << "Queue is empty" << std::endl;
else
    std::cout << "Queue is not empty" << std::endl;
std::cout << "=====\n\n";

// size()
std::cout << "Queue size is: " << q.size() << std::endl;
std::cout << "=====\n\n";
// front
std::cout << "Front Element: " << q.front() << std::endl;
std::cout << "=====\n\n";
// back
std::cout << "Back Element: " << q.back() << std::endl;
std::cout << "=====\n\n";
// Display
std::cout << "Queue contents:";
display(q);
std::cout << "=====\n\n";
// Push another element
q.push('X');
// Display the updated queue
std::cout << "After pushing 'X':" ;
display(q);
std::cout << "=====\n\n";

std::cout << "Deleting all elements....\n";
// deletion
while (!q.empty())
{
    display(q);
    q.pop();
}
```

```
std::cout << "Final Queue:" << std::endl;
display(q);

return 0;
}

void display(std::queue<char> r)
{
    // Create a copy of the queue
    std::queue<char> c = r;

    while (!c.empty())
    {
        std::cout << " " << c.front();
        c.pop();
    }

    std::cout << std::endl;
}
```

Output

```
=====
Queue Implementation
=====

Display:  c p e 0 1 0
=====

Queue is not empty
=====

Queue size is: 6
=====

Front Element: c
=====

Back Element: 0
=====

Queue contents: c p e 0 1 0
=====

After pushing 'X': c p e 0 1 0 X
=====

Deleting all elements....
c p e 0 1 0 X
p e 0 1 0 X
e 0 1 0 X
0 1 0 X
1 0 X
0 X
X
Final Queue:
```

Code Analysis

This C++ program shows how a Queue data structure works using basic operations like adding, displaying, checking, and removing elements. It uses `push()` to add data, `front()` and `back()` to access elements, and `pop()` to remove items following the FIFO (First In, First Out) principle. The program also uses a `display` function to show the queue contents without changing the original data.

ILOB.1	
Function	Source Code/Header File
Creating new node and public variables	<pre> template <typename T> class qNode { public: T data; qNode<T> *next; }; // Create a new node template <typename T> qNode<T> *new_node(T newData) { qNode<T> *newNode = new qNode<T>; newNode->data = newData; newNode->next = nullptr; return newNode; } </pre>
Enqueue the elements	<pre> // Enqueue template <typename T> void enqueue(qNode<T> **frontPtr, qNode<T> **backPtr, T newData) { // Create new node qNode<T> *newNode = new_node(newData); // Queue is empty if ((*frontPtr) == nullptr && (*backPtr) == nullptr) { *frontPtr = newNode; *backPtr = newNode; return; } // Queue is not empty (*backPtr)->next = newNode; *backPtr = newNode; } </pre>

Dequeue the elements

```
// Dequeue
template <typename T>
void dequeue(qNode<T> **frontPtr, qNode<T> **backPtr)
{
    // Queue is empty
    if (*frontPtr == nullptr)
    {
        return;
    }

    // Temporary node
    qNode<T> *deleteNode = *frontPtr;

    // Only one node
    if ((*frontPtr)->next == nullptr)
    {
        *frontPtr = nullptr;
        *backPtr = nullptr;
        delete deleteNode;
        return;
    }

    // More than one node
    *frontPtr = (*frontPtr)->next;
    delete deleteNode;
}
```

Display all the elements

```
// display all the elements
template <typename T>
void display(qNode<T> *frontPtr)
{
    qNode<T> *temp = frontPtr;

    if (temp == nullptr)
    {
        std::cout << "Queue is empty.\n";
        return;
    }

    while (temp != nullptr)
    {
        std::cout << temp->data << " \n";
        temp = temp->next;
    }
}
```

Return or print the front element

```
// return the front variable
template <typename T>
T frontFind (qNode<T> *frontPtr) {
    return frontPtr->data;
}
```

Code Analysis

The header file contains the functions and classes needed to create and manage the queue. The qNode class defines each node that stores data and a pointer to the next node. The new_node() function creates a new node, while enqueue() adds new elements at the back of the queue. The dequeue() function removes elements from the front and updates the pointers properly. The display() function shows all elements inside the queue by going through each node, and frontFind() returns the value of the first element. Overall, the header file provides the functions needed to implement a queue using a linked list.

Main Code

```
#include <iostream>
#include "queuel1.h"

int main()
{
    qNode<char> *front = nullptr;
    qNode<char> *back = nullptr;

    enqueue(&front, &back, 'R');
    std::cout << "Front: " << front->data << " Back: " << back->data << std::endl;

    enqueue(&front, &back, 'Y');
    std::cout << "Front: " << front->data << " Back: " << back->data << std::endl;

    enqueue(&front, &back, 'O');
    std::cout << "Front: " << front->data << " Back: " << back->data << std::endl;

    std::cout << "\nDequeue (R)" << std::endl;
    dequeue(&front, &back);

    std::cout << "Front: " << front->data
    << " Back: " << back->data << std::endl;

    std::cout << "\nDisplay all: " << std::endl;
    display(front);

    std::cout << "Front Element: " << frontFind(front) << std::endl;

    return 0;
}
```

Output

```
Front: R Back: R
Front: R Back: Y
Front: R Back: O

Dequeue (R)
Front: Y Back: O

Display all:
Y
O
Front Element: Y
```


Code Analysis

The main file controls the execution of the queue program. It creates an empty queue using two pointers, front and back, which represent the first and last elements of the queue. The program adds elements using the enqueue() function and displays the current front and back values after each insertion. It then removes the first element using dequeue() and shows the updated queue. The display() function is used to print all remaining elements, while frontFind() is used to get the first element of the queue. Overall, the main file tests and demonstrates the basic operations of a queue.

ILOB.2

Header File

```
#ifndef QUEUEARRAY_H
#define QUEUEARRAY_H

#include <iostream>
using namespace std;

class Queue
{
private:
    int* q_array;
    int q_capacity;
    int q_size;
    int q_front;
    int q_back;

public:
    // Constructor
    Queue(int capacity = 10)
    {
        q_capacity = capacity;
        q_array = new int[q_capacity];
        q_size = 0;
        q_front = 0;
```

```
        q_back = -1;
    }

    // Checks if queue is empty
    bool Empty() const
    {
        return q_size == 0;
    }

    // Returns the number of elements
    int Size() const
    {
        return q_size;
    }

    // Clears the queue
    void Clear()
    {
        q_size = 0;
        q_front = 0;
        q_back = -1;
    }

    // Returns the front element
    int Front() const
    {
        if (Empty())
        {
            cout << "Queue is empty." << endl;
            return -1;
        }

        return q_array[q_front];
    }

    // Returns the back element
```

```
int Back() const
{
    if (Empty())
    {
        cout << "Queue is empty." << endl;
        return -1;
    }

    return q_array[q_back];
}

// Adds an element to the queue
void Enqueue(int value)
{
    if (q_size == q_capacity)
    {
        cout << "Queue is full." << endl;
        return;
    }

    q_back = (q_back + 1) % q_capacity;
    q_array[q_back] = value;
    q_size++;
}

// Removes the front element
void Dequeue()
{
    if (Empty())
    {
        cout << "Queue is empty." << endl;
        return;
    }

    q_front = (q_front + 1) % q_capacity;
    q_size--;
```

```
}

// Copy Constructor
Queue(const Queue& other)
{
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;

    q_array = new int[q_capacity];

    for (int i = 0; i < q_capacity; i++)
    {
        q_array[i] = other.q_array[i];
    }
}

// Copy Assignment Operator
Queue& operator=(const Queue& other)
{
    if (this != &other)
    {
        delete[] q_array;

        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;

        q_array = new int[q_capacity];

        for (int i = 0; i < q_capacity; i++)
        {
            q_array[i] = other.q_array[i];
        }
    }
}
```

```
    }

    return *this;
}

// Destructor
~Queue()
{
    delete[] q_array;
}
};

#endif
```

Main Code

```
#include <iostream>
#include "queuearray.h"
using namespace std;

int main()
{
    Queue q(5);

    cout << "=== QUEUE TEST ===" << endl;

    // Test Empty()
    cout << "Is queue empty? "
         << (q.Empty() ? "Yes" : "No") << endl;

    // Test Enqueue()
    cout << "\nAdding elements: 10, 20, 30" << endl;
    q.Enqueue(10);
    q.Enqueue(20);
    q.Enqueue(30);
```

```
// Test Size()
cout << "Queue size: " << q.Size() << endl;

// Test Front()
cout << "Front element: " << q.Front() << endl;

// Test Back()
cout << "Back element: " << q.Back() << endl;

// Test Dequeue()
cout << "\nRemoving front element..." << endl;
q.Dequeue();

cout << "New front: " << q.Front() << endl;
cout << "Queue size: " << q.Size() << endl;

// Test Copy Constructor
cout << "\nTesting Copy Constructor..." << endl;
Queue q2(q);

cout << "Copied queue front: " << q2.Front() << endl;
cout << "Copied queue back: " << q2.Back() << endl;

// Test Assignment Operator
cout << "\nTesting Copy Assignment Operator..." << endl;
Queue q3;
q3 = q;

cout << "Assigned queue front: " << q3.Front() << endl;
cout << "Assigned queue back: " << q3.Back() << endl;

// Test Clear()
cout << "\nClearing original queue..." << endl;
q.Clear();

cout << "Queue size: " << q.Size() << endl;
```

```
cout << "Is queue empty? "  
      << (q.Empty() ? "Yes" : "No") << endl;  
  
return 0;  
}
```

Code Analysis

The source code creates a Queue class using a dynamically allocated array to store elements. It includes the required queue functions such as Enqueue() for adding elements, Dequeue() for removing elements, Front() and Back() for checking the first and last elements, Size() for checking the number of elements, and Empty() and Clear() for checking and clearing the queue. It also includes a copy constructor, copy assignment operator, and destructor to properly copy and manage the queue in memory.

Output

```
; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }  
=== QUEUE TEST ===  
Is queue empty? Yes  
  
Adding elements: 10, 20, 30  
Queue size: 3  
Front element: 10  
Back element: 30  
  
Removing front element...  
New front: 20  
Queue size: 2  
  
Testing Copy Constructor...  
Copied queue front: 20  
Copied queue back: 30  
  
Testing Copy Assignment Operator...  
Assigned queue front: 20  
Assigned queue back: 30  
  
Clearing original queue...  
Queue size: 0  
Is queue empty? Yes
```

Code Analysis

The output shows that the queue functions work correctly. The program first checks that the queue is empty, then adds 10, 20, and 30, making the size 3 with 10 as the front and 30 as the back. After removing the front element, 20 becomes the new front and the size becomes 2. The copy constructor and assignment operator are also tested successfully. Finally, the Clear() function removes all elements, making the queue size 0 and confirming that the queue is empty.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILOC

Header File

```
#ifndef PRINTER_H
#define PRINTER_H

#include <iostream>
#include <string>
using namespace std;

// Task 1: Job class
class Job {
public:
    int id;
    string user;
    int pages;

    Job(int i, string u, int p) {
        id = i;
        user = u;
        pages = p;
    }
};

// Node for the linked list
class Node {
```



```
public:
    Job* job;
    Node* next;

    Node(Job* j) {
        job = j;
        next = nullptr;
    }
};

// Task 2 and 3: Printer class
class Printer {
private:
    Node* front;
    Node* rear;

public:
    Printer() {
        front = nullptr;
        rear = nullptr;
    }

    // Add a job
    void addJob(Job* job) {
        Node* newNode = new Node(job);

        if (rear == nullptr) {
            front = rear = newNode;
        } else {
            rear->next = newNode;
            rear = newNode;
        }

        cout << "Job " << job->id << " added by "
              << job->user << " (" << job->pages
              << " pages)" << endl;
    }
};
```

```
// Process jobs
void processJobs() {
    cout << "\nProcessing Jobs:" << endl;

    while (front != nullptr) {
        cout << "Printing Job " << front->job->id
            << " - " << front->job->user
            << " (" << front->job->pages
            << " pages)" << endl;

        Node* temp = front;
        front = front->next;
        delete temp;
    }

    rear = nullptr;
}
};

#endif
```

Main Code

```
#include <iostream>
#include "Printer.h"
using namespace std;

int main() {
    Printer printer;

    // Task 4: Create different jobs
    Job job1(1, "Ryo", 8);
    Job job2(2, "Khaira", 6);
    Job job3(3, "Jabez", 7);
    Job job4(4, "Ivan", 9);
```

```
// Add jobs to the printer
printer.addJob(&job1);
printer.addJob(&job2);
printer.addJob(&job3);
printer.addJob(&job4);

// Process all jobs
printer.processJobs();

return 0;
}
```

Output

```
?) { g++ main.cpp -o main } ; if ($?) { .\main }
Job 1 added by Ryo (8 pages)
Job 2 added by Khaira (6 pages)
Job 3 added by Jabez (7 pages)
Job 4 added by Ivan (9 pages)

Processing Jobs:
Printing Job 1 - Ryo (8 pages)
Printing Job 2 - Khaira (6 pages)
Printing Job 3 - Jabez (7 pages)
Printing Job 4 - Ivan (9 pages)
```

Code Analysis

The output is in the same order as the jobs were added. Alice's job is printed first because she submitted it first, followed by Bob, Charlie, and David. This follows the FIFO (First In, First Out) rule of a queue. The linked list stores each job in a node and allows new jobs to be added at the rear and processed from the front.

Conclusion

Overall, I learned how a queue works using the FIFO (First In, First Out) principle and how to implement it using a linked list. I learned how to create classes, add jobs to the queue, and process them in the correct order. I also learned how front, rear, nodes, and pointers work together to manage the queue. This activity helped me understand how queues can be used in real-life situations, such as managing print jobs in a shared printer.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT, QuillBot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code Assistance, Grammar Correction

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I confirm that Ai did not replace my critical thinking.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.